

JAVA 17 OCP · YDS STUDY SERIES

Unit 13 · Concurrency — Technical Memory Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu not, [ana çift dilli dersin](#) Java 17 thread, executor, thread-safety ve parallel stream kurallarını sınav kararı biçiminde sıkıştırır. Sorular özgün OCP tarzı çalışma sorularıdır; gerçek sınav sorusu değildir.

1. Process, thread ve task

```
process
├─ shared heap / static state
├─ thread 1 → aynı anda bir task yürütür
├─ thread 2 → aynı anda bir task yürütür
└─ thread n → aynı anda bir task yürütür
```

- **Process:** Kendi runtime environment'ında çalışan program.
- **Thread:** OS scheduler tarafından planlanabilen execution unit.
- **Task:** Bir thread'in gerçekleştirdiği tek iş; Java'da çoğunlukla `Runnable` veya `Callable`.
- **Concurrency:** Task'ların zaman aralıkları çakışacak biçimde ilerlemesi.
- **Parallelism:** Task'ların gerçekten aynı anda farklı CPU core'larında yürüyebilmesi.

Concurrency parallelism'i mümkün kılar; fakat tek CPU'da context switch ile de concurrent execution görülebilir.

2. Scheduler, context switch ve nondeterminism

Thread scheduler hangi RUNNABLE thread'in ne zaman çalışacağını garanti etmez. Bir thread'in state'ini kaydedip diğerine geçmeye **context switch** denir.

```
new Thread() -> System.out.print("A").start();
System.out.print("B");
```

Possible output:

```
AB
```

veya:

```
BA
```

Kod derlenir; output order nondeterministic'tir. Bir makinede sürekli aynı sonucu gözlemlemek language guarantee oluşturmaz.

3. Runnable ve Callable

Özellik	<code>Runnable</code>	<code>Callable<V></code>
Abstract method	<code>void run()</code>	<code>V call() throws Exception</code>
Return value	Yok	Var
Checked exception	Declare edemez	Declare edebilir
Lambda parameter	0	0

```
Runnable r = () -> System.out.println("clean");
Callable<Integer> c = () -> 42;
```

`Runnable` ve `Callable` functional interface'tir. Lambda body normal Java kurallarına uymalıdır:

```
Callable<Integer> bad = () -> { 42; }; // DOES NOT COMPILE
```

Doğrusu:

```
Callable<Integer> good = () -> { return 42; };
```

4. Thread oluşturmak

```
Thread worker = new Thread(
    () -> System.out.println(Thread.currentThread().getName())
);
worker.start();
```

`start()` yeni execution path planlar ve JVM uygun zamanda `run()` çağırır.

```
Thread worker = new Thread(() -> System.out.print("A"));
worker.run();
System.out.print("B");
```

Output kesin olarak **AB**dir. `run()` normal method call'dır; yeni thread başlatmaz.

OCP trap: `start()` concurrency; doğrudan `run()` synchronous call.

5. Aynı Thread'i iki kez başlatmak

```
Thread t = new Thread(() -> {});
t.start();
t.start();
```

Kod derlenir. İkinci `start()` çağırısı runtime'da `IllegalThreadStateException` fırlatır. Terminated thread yeniden başlatılamaz; yeni `Thread` object'i gerekir.

6. Thread lifecycle

Java 17 `Thread.State` değerleri:

State	Anlam
NEW	<code>start()</code> henüz çağırılmadı
RUNNABLE	Çalışıyor veya CPU bekliyor
BLOCKED	Intrinsic monitor lock bekliyor

State	Anlam
WAITING	Süresiz başka eylem bekliyor
TIMED_WAITING	Belirli süre bekliyor
TERMINATED	<code>run()</code> tamamlandı

Java enum'unda ayrı bir `RUNNING` state yoktur. `getState()` yalnız anlık snapshot verir; hemen sonra değişebilir.

Memory tip: `NEW` → `RUNNABLE` → `waiting/blocked states` → `TERMINATED`.

7. `sleep()` ve polling

```
while (inventory.isEmpty()) {
    Thread.sleep(100);
}
```

`Thread.sleep()`:

- `static` method'dur; current thread'i bekletir.
- `InterruptedException` declare eder.
- Thread'i `TIMED_WAITING` state'ine geçirebilir.
- Süre dolunca hemen CPU almayı garanti etmez.
- Lock'u otomatik bırakmaz.

Busy waiting:

```
while (inventory.isEmpty()) {
    // CPU tüketir
}
```

Polling'e `sleep` eklemek CPU kullanımını azaltır; yine de higher-level coordination primitive daha uygun olabilir.

8. Interrupt davranışı

```
Thread t = new Thread(() -> {
    try {
        Thread.sleep(10_000);
    } catch (InterruptedException e) {
        System.out.println("interrupted");
    }
});
t.start();
t.interrupt();
```

Sleeping/waiting thread interrupt edilirse `InterruptedException` alabilir ve interrupt status temizlenir.

Method	Hangi thread?	Flag'i temizler mi?
<code>t.interrupt()</code>	<code>t</code> 'ye signal gönderir	Doğrudan sorgu değil
<code>t.isInterrupted()</code>	<code>t</code> 'nin status'u	Hayır
<code>Thread.interrupted()</code>	Current thread	Evet

Interrupt zorla thread öldürme değildir. Task signal'ı kontrol etmiyor ve interruptible blocking call'da değilse çalışmaya devam edebilir.

9. Concurrency API neden tercih edilir?

Raw `Thread` oluşturmak task ile thread management'ı birbirine bağlar. `ExecutorService`:

- Thread lifecycle'ı yönetir.
- Task submission ve scheduling policy'sini concrete implementation belirler.
- Thread reuse eder.
- Pool size ile concurrency'yi sınırlar.
- `Future` üzerinden sonuç/hata takibi sunar.

```
ExecutorService service = Executors.newSingleThreadExecutor();
try {
    service.execute(() -> System.out.println("task"));
} finally {
    service.shutdown();
}
```

10. Single-thread executor ve pool'lar

Factory	Davranış
<code>newSingleThreadExecutor()</code>	Tek worker; task'lar unbounded queue üzerinden sıralı yürür
<code>newFixedThreadPool(n)</code>	En fazla <code>n</code> worker; ek task'lar unbounded queue'da bekler
<code>newCachedThreadPool()</code>	Idle worker yoksa yeni thread oluşturabilir; klasik waiting queue davranışı yoktur
<code>newSingleThreadScheduledExecutor()</code>	Tek scheduled worker
<code>newScheduledThreadPool(n)</code>	Scheduled task pool

Single-thread executor içinde task'lar aynı anda çalışmaz. Bu, dışarıdaki başka thread'lerin shared state'e erişmediği anlamına gelmez.

11. `execute()` ve `submit()`

Method	Input	Return
<code>execute(Runnable)</code>	<code>Runnable</code>	<code>void</code>
<code>submit(Runnable)</code>	<code>Runnable</code>	<code>Future<?> (get() → null)</code>
<code>submit(Runnable, T)</code>	<code>Runnable + fixed result</code>	<code>Future<T></code>
<code>submit(Callable<T>)</code>	<code>Callable<T></code>	<code>Future<T></code>

```
Future<Integer> value = service.submit(() -> 42);
System.out.println(value.get()); // 42
```

```
Future<?> bad = service.execute(() -> {}); // DOES NOT COMPILE
```

OCP trap: `execute()` return type'i `void`dur.

12. Future

Common methods:

```
V get()
V get(long timeout, TimeUnit unit)
boolean cancel(boolean mayInterruptIfRunning)
boolean isCancelled()
boolean isDone()
```

Exception haritası:

Durum	Sonuç
Task normal tamamlanır	<code>get()</code> value/null döndürür
Task exception ile biter	<code>get()</code> → <code>ExecutionException</code>
Waiting thread interrupt edilir	<code>InterruptedException</code>
Timed get süresi dolar	<code>TimeoutException</code>
Future cancel edilmiştir	<code>CancellationException</code>

`isDone()`; normal, exception veya cancellation ile completion sonrasında `true` olabilir. “Başarılı tamamlandı” ile aynı şey değildir.

OCP trap: Timed `get()` süresinin dolması yalnız bekleyen çağrının `TimeoutException` ile dönmesini sağlar. Çalışan task'ı cancel veya interrupt etmez.

13. Executor shutdown

```
service.shutdown();
```

- Yeni task kabul etmez.
- Daha önce submitted task'ların tamamlanmasına izin verir.
- Running task'ları interrupt etmez.
- Çağrı hemen döner.

```
List<Runnable> queued = service.shutdownNow();
```

- Running task'ları interrupt etmeye çalışır.
- Başlamamış queued task'ları döndürür.
- Task'ların kesin olarak durduğunu garanti etmez.

```
service.shutdown();
if (!service.awaitTermination(5, TimeUnit.SECONDS)) {
    service.shutdownNow();
}
```

Executor kapatılmazsa non-daemon worker thread'ler JVM'in sona ermesini engelleyebilir.

14. ScheduledExecutorService

Tek seferlik task:

```
ScheduledFuture<Integer> f =
    scheduler.schedule(() -> 42, 1, TimeUnit.SECONDS);
```

Fixed rate:

```
scheduler.scheduleAtFixedRate(
    task, 0, 5, TimeUnit.SECONDS
);
```

Fixed delay:

```
scheduler.scheduleWithFixedDelay(
    task, 0, 5, TimeUnit.SECONDS
);
```

Method	Bir sonraki başlangıç
<code>scheduleAtFixedRate</code>	Önceki planned start'a göre period
<code>scheduleWithFixedDelay</code>	Önceki completion'dan sonra delay

Periodic method'lar `Runnable` alır, `Callable` değil. Task süresi veya pool capacity actual timing'i etkiler.

15. Thread-safety üçlüsü

Shared state analizinde üç kavramı ayır:

- 1 **Atomicity:** Operation bölünemez mi?
- 2 **Visibility:** Bir thread'in write'ını diğer thread ne zaman görür?
- 3 **Ordering:** Compiler/CPU reorder etkileri uygun happens-before ile sınırlandı mı?

Thread-safe code, valid state invariant'larını bütün interleaving'lerde korur.

16. Race condition

```
private int sheepCount;

void increment() {
    sheepCount++;
}
```

`++` tek adım değildir:

```
read → add 1 → write
```

İki thread aynı eski value'yu okuyup update'lerden birini kaybedebilir. 100 task sonunda sonuç 1–100 aralığında olabilir; tam değer garanti edilmez.

Memory tip: “Tek satır, tek atomic operation demek değildir.”

17. `synchronized`

Instance method:

```
synchronized void increment() {
    sheepCount++;
}
```

Lock object: current instance (`this`).

Static method:

```
static synchronized void reset() {
    count = 0;
}
```

Lock object: `Class` object (`TypeName.class`).

Block:

```
void increment() {
    synchronized (this) {
        sheepCount++;
    }
}
```

Intrinsic monitor reentrant'tır: Aynı thread sahip olduğu monitor'a tekrar girebilir. Instance ve static synchronized method farklı lock'lar kullandığı için birbirini otomatik korumaz.

18. Synchronized collection wrapper

```
List<Integer> list =
    Collections.synchronizedList(new ArrayList<>());
```

Individual method call'lar wrapper lock'u ile korunur. Compound iteration için manual synchronization gerekir:

```
synchronized (list) {
    for (Integer value : list) {
        System.out.println(value);
    }
}
```

Wrapper, iteration'ı otomatik atomic transaction'a dönüştürmez.

19. Atomic class'lar

```
AtomicInteger count = new AtomicInteger();
System.out.println(count.incrementAndGet()); // 1
System.out.println(count.getAndIncrement()); // 1
System.out.println(count.get());             // 2
```

Method	Return
<code>incrementAndGet()</code>	Yeni value
<code>getAndIncrement()</code>	Eski value
<code>addAndGet(n)</code>	Yeni value
<code>getAndAdd(n)</code>	Eski value
<code>compareAndSet(expected, update)</code>	Başarı Boolean

Tek atomic method güvenlidir; birden fazla atomic call'dan oluşan compound invariant kendiliğinden atomic değildir.

```
if (count.get() < 10) {
    count.incrementAndGet();
}
```

Bu check-then-act bütünü race condition içerebilir.

20. `volatile`

```
private volatile boolean running = true;
```

`volatile`, write/read arasında visibility ve ordering garantileri sağlar. Compound update'ı atomic yapmaz:

```
private volatile int count;

void increment() {
    count++; // hâlâ race condition
}
```

İhtiyaç	Uygun seçenek
Basit status flag visibility	<code>volatile</code> olabilir
Atomic counter	<code>AtomicInteger</code>
Birden çok field invariant'ı	Lock / synchronized

21. Lock framework

```
Lock lock = new ReentrantLock();
lock.lock();
try {
    updateState();
} finally {
    lock.unlock();
}
```

`unlock()` her zaman `finally` içinde olmalıdır. `lock()` lock elde edilene kadar bekleyebilir.

Nonblocking attempt:

```
if (lock.tryLock()) {
    try {
        updateState();
    } finally {
        lock.unlock();
    }
}
```

`tryLock()` `false` dönerse protected section'a girilmez ve `unlock()` çağrılmaz. Sahip olunmayan lock'u unlock etmek runtime `IllegalMonitorStateException` üretebilir.

22. Timed lock ve fairness

```
if (lock.tryLock(1, TimeUnit.SECONDS)) {
    try {
        updateState();
    } finally {
        lock.unlock();
    }
}
```

Timed overload `InterruptedException` declare eder.

```
Lock fair = new ReentrantLock(true);
```

Fair lock waiting thread'lere yaklaşık arrival order önceliği verir; mutlak scheduling sırası garantisi değildir ve throughput maliyeti olabilir. `ReentrantLock` aynı owner thread'in birden çok kez lock etmesine izin verir; her başarılı acquisition için uygun sayıda `unlock()` gerekir.

23. Concurrent collection'lar

Type	Temel özellik
<code>ConcurrentHashMap</code>	Concurrent map; null key/value kabul etmez
<code>ConcurrentSkipListMap</code>	Concurrent sorted/navigable map
<code>ConcurrentSkipListSet</code>	Concurrent sorted/navigable set
<code>CopyOnWriteArrayList</code>	Her write'da copy; iterator snapshot görür
<code>CopyOnWriteArraySet</code>	Copy-on-write set
<code>LinkedBlockingQueue</code>	Blocking queue operation'ları

```
var list = new CopyOnWriteArrayList<>(List.of(1, 2, 3));
for (int value : list) {
    list.add(value + 10);
}
System.out.println(list.size()); // 6
```

Iterator başlangıç snapshot'ını gördüğü için loop üç kez çalışır.

24. CyclicBarrier

```
var barrier = new CyclicBarrier(
    3, () -> System.out.println("all ready")
);

Runnable task = () -> {
    prepare();
    try {
        barrier.await();
        continueWork();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } catch (BrokenBarrierException e) {
        throw new IllegalStateException(e);
    }
};
```

- Constructor'daki parties sayısı kadar thread `await()` e gelince barrier action bir kez çalışır.
- Sonra waiting thread'ler devam eder.
- Barrier cycle sonrasında yeniden kullanılabilir.
- Yeterli task/worker yoksa program sonsuza kadar bekleyebilir.
- Barrier action'ı parties'den biri tarafından, release öncesinde çalıştırılır.

OCP trap: Parallel stream'in barrier parties kadar worker tahsis edeceği garanti değildir.

25. Liveness problem'ları

Deadlock

İki thread birbirinin tuttuğu lock'u sonsuza kadar bekler:

```
T1 holds A → waits B
T2 holds B → waits A
```

Starvation

Bir thread, sürekli diğerleri kaynak aldığı için uzun süre/süresiz ilerleyemez.

Livelock

Thread'ler blocked değildir; birbirine tepki vererek state değiştirir fakat faydalı ilerleme sağlayamaz.

Problem	Thread'ler aktif mi?	İlerleme
Deadlock	Bekliyor/blocked	Yok
Starvation	En az biri kaynak alamıyor	O thread için yok
Livelock	Aktif biçimde tepki veriyor	Yok

Consistent global lock order deadlock riskini azaltır.

26. Parallel stream oluşturmak

```
Stream<Integer> a = list.parallelStream();  
Stream<Integer> b = list.stream().parallel();
```

Sequential'a dönmek:

```
Stream<Integer> c = a.sequential();
```

Son çağrılan execution-mode operation pipeline mode'unu belirler. Parallel stream her zaman daha hızlı değildir; data size, splitting cost, operation ve core count önemlidir.

27. Ordering: `forEach`, `forEachOrdered`, `findAny`

```
List.of(1, 2, 3, 4)  
    .parallelStream()  
    .forEach(System.out::print);
```

Element order garanti edilmez.

```
List.of(1, 2, 3, 4)  
    .parallelStream()  
    .forEachOrdered(System.out::print);
```

Encounter order varsa output `1234` olur; ordering parallel performance'i sınırlayabilir.

`findFirst()` encounter order'daki first element'i korur. `findAny()` herhangi bir uygun element döndürebilir; sequential stream'de sıkça first görünse bile garanti değildir.

28. Parallel reduction şartları

Safe reduction:

```
int sum = List.of(1, 2, 3, 4)  
    .parallelStream()  
    .reduce(0, Integer::sum);
```

Sonuç `10`dur.

Reduction için:

- Identity gerçekten neutral olmalı.
- Accumulator stateless ve associative olmalı.
- Combiner partial result'ları aynı mantıkla birleştirmeli.
- Accumulator/combiner compatible olmalı.
- Shared mutable side effect kullanılmamalı.

Subtraction associative değildir:

```
int result = stream.parallel().reduce(0, (a, b) -> a - b);
```

Grouping'e göre farklı sonuçlar çıkabilir.

29. Three-argument `reduce`

```
int totalLength = List.of("wolf", "bear")
    .parallelStream()
    .reduce(
        0,
        (length, word) -> length + word.length(),
        Integer::sum
    );
```

Argument	Type rolü
Identity	Result type başlangıcı
Accumulator	Result + element → result
Combiner	Result + result → result

Combiner parallel pipeline'da kritiktir; sequential gözlemede çağrılmaması onun yanlış olabileceği anlamına gelmez.

30. Parallel `collect`

Mutable reduction:

```
List<String> result = stream.parallel().collect(
    ArrayList::new,
    ArrayList::add,
    ArrayList::addAll
);
```

Framework farklı partition'lar için ayrı result container'ları oluşturup combiner ile birleştirebilir.

`Collector.Characteristics.CONCURRENT` tek başına yeterli değildir; concurrent accumulation için collector ve stream ordering özellikleri birlikte değerlendirilir. Built-in collector'lar mümkün olduğunda tercih edilir.

31. Stateful lambda ve side effect tuzağı

```
List<Integer> output = new ArrayList<>();
source.parallelStream().forEach(output::add);
```

`ArrayList` concurrent write için thread-safe değildir. Runtime'da missing element, data corruption veya exception görülebilir; sonuç garanti edilmez.

Doğrusu:

```
List<Integer> output = source.parallelStream().toList();
```

Stream operation'larında kullanılan lambda'lar stateless ve non-interfering olmalıdır. Associativity şartı ise parallel reduction'daki accumulator ve combiner gibi partial result'ları birleştiren operation'lar için geçerlidir; `map`, `filter` veya `forEach` lambda'ları için genel bir şart değildir.

32. Compile-time, runtime ve output ayrımı

Durum	Sınıflandırma
<code>Future<?> f = service.execute(r)</code>	Does not compile
<code>Runnable r = () -> { return 1; }</code>	Does not compile
Aynı Thread'e ikinci <code>start()</code>	Runtime <code>IllegalThreadStateException</code>
Sahip olunmayan <code>ReentrantLock.unlock()</code>	Runtime <code>IllegalMonitorStateException</code>
<code>Future.get()</code> failed task	Runtime <code>ExecutionException</code>
Single-thread executor'da iki submitted print	Submission order'ında yürür
Parallel <code>forEach</code>	Derlenir; order nondeterministic
Barrier parties task sayısından büyük	Derlenir; runtime hang olabilir
Executor kapatılmıyor	Work bitebilir; JVM terminate etmeyebilir

33. OCP çözüm algoritması

- 1 Task `Runnable` mı `Callable` mı? Return/checked exception kontrol et.
- 2 `run()` mı `start()` mı çağırılmış?
- 3 Executor factory pool size kaç?
- 4 `execute/submit` overload ve return type uyumlu mu?
- 5 Executor kesin kapatılıyor mu?
- 6 Shared mutable state var mı?
- 7 Her read-modify-write atomic mi?
- 8 `synchronized`, `atomic`, `volatile` veya Lock hangi garantiyi sağlıyor?
- 9 Aynı data aynı lock ile mi korunuyor?
- 10 `tryLock()` return value kontrol edilip yalnız success'te unlock ediliyor mu?
- 11 Lock acquisition order cycle oluşturabilir mi?
- 12 Barrier parties ile ulaşabilecek worker/task sayısı uyumlu mu?
- 13 Stream parallel mı; encounter order gerekli mi?
- 14 Reduction identity/accumulator/combiner associative ve compatible mı?
- 15 Sonucu **Does not compile**, runtime exception, hang veya allowed output set'i olarak sınıflandır.

34. Mini quiz · Özgün OCP tarzı çalışma soruları

Soru 1

```
Thread t = new Thread(() -> System.out.print("A"));
t.run();
System.out.print("B");
```

Output nedir?

Soru 2

```
Thread t = new Thread(() -> {});
t.start();
t.start();
```

Compile-time mı runtime mı hata oluşur?

Soru 3

Hangileri geçerli functional interface assignment'ıdır? İki seçeneği seçin.

- A. `Runnable a = () -> 1;`
- B. `Callable<Integer> b = () -> 1;`
- C. `Runnable c = () -> System.out.println("x");`
- D. `Callable<Integer> d = x -> x + 1;`

Soru 4

```
Future<?> f = service.execute(() -> {});
```

Sonuç nedir?

Soru 5

`submit(Runnable)` ile dönen `Future<?>` normal completion sonrasında `get()` çağrısında ne döndürür?

Soru 6

```
volatile int count;
void increment() { count++; }
```

Bu code thread-safe midir?

Soru 7

```
if (lock.tryLock()) {
    work();
}
lock.unlock();
```

Temel runtime riski nedir?

Soru 8

Instance `synchronized` method ile static `synchronized` method aynı lock'u mu kullanır?

Soru 9

Üç task `newSingleThreadExecutor()` a 1, 2, 3 sırasıyla submit ediliyor. Başka exception yok. Execution order nedir?

Soru 10

Bir `CyclicBarrier(5)` e ulaşabilecek yalnız dört worker vardır. Beşinci çağrı hiç yapılmazsa en olası liveness sonucu nedir?

Soru 11

```
List.of(1, 2, 3, 4)
    .parallelStream()
    .forEach(System.out::print);
```

1234 guaranteed midir?

Soru 12

```
int n = List.of(1, 2, 3, 4)
    .parallelStream()
    .reduce(0, Integer::sum);
```

Sonuç nedir ve neden güvenlidir?

Soru 13

Subtraction parallel reduction için neden risklidir?

Soru 14

`shutdown()` ile `shutdownNow()` arasındaki temel farkı açıklayın.

Soru 15

Bir thread lock A'yı tutup B'yi, diğer thread B'yi tutup A'yı bekliyor. Problem adı nedir?

Soru 16

`Thread.interrupted()` ile `someThread.isInterrupted()` arasındaki flag farkı nedir?

Cevaplar ve açıklamalar

- 1 **AB.** `run()` normal synchronous method call'dır. **A** tamamlandıktan sonra main thread **B** yazar.
- 2 Kod derlenir; ikinci `start()` runtime `IllegalThreadStateException` üretir.
- 3 **B ve C.** `Callable<Integer>` value döndürür; `Runnable` void-compatible body alır. D seçeneği bir parameter aldığı için `Callable` signature'ına uymaz.
- 4 **Does not compile.** `execute(Runnable)` return type'i `void`dur.
- 5 **null.** `Runnable` bir result üretmez. `submit(Runnable, T)` overload'ı kullanılsaydı verilen fixed result dönebilirdi.

- 6 Hayır. `volatile` visibility sağlar; `count++` read-modify-write sequence'ını atomic yapmaz.
- 7 `tryLock()` `false` döndüğünde thread lock sahibi değildir; koşulsuz `unlock()` runtime `IllegalMonitorStateException` üretebilir.
- 8 Hayır. Instance method `this` monitor'ını, static method `Class` object monitor'ını kullanır.
- 9 1, 2, 3. Single-thread executor submitted task'ları tek worker üzerinde sırayla yürütür.
- 10 Barrier trip olmaz; dört worker runtime'da süresiz bekleyebilir. Bu bir hang/liveness failure'dır.
- 11 Hayır. Parallel `forEach` encounter order'ı korumaz. 1234 allowed olabilir fakat guaranteed değildir.
- 12 10. 0 addition identity'sidir ve integer addition associative'tir; partial sums güvenle birleştirilebilir.
- 13 $(a-b)-c$, $a-(b-c)$ ye eşit değildir. Operation associative olmadığı için partition/grouping değişince sonuç değişebilir.
- 14 `shutdown()` yeni task kabul etmez, queued/running task'ların bitmesine izin verir. `shutdownNow()` running task'ları interrupt etmeye çalışır ve başlamamış queued task'ları döndürür; kesin durdurma garantisi vermez.
- 15 **Deadlock.** Her thread diğerinin bırakmayacağı lock'u bekler.
- 16 `Thread.interrupted()` current thread'in flag'ini okuyup temizler. `someThread.isInterrupted()` belirtilen thread'in flag'ini okur fakat temizlemez.

Son tekrar kartı

<code>start()</code>	new execution
<code>run()</code>	normal method call
<code>Runnable</code>	void, checked exception yok
<code>Callable<V></code>	value, checked exception olabilir
<code>execute</code>	void
<code>submit</code>	<code>Future</code>
<code>shutdown</code>	orderly
<code>shutdownNow</code>	interrupt attempt + queued list
<code>volatile</code>	visibility, NOT ++ atomicity
<code>AtomicInteger</code>	single atomic operations
<code>synchronized</code>	intrinsic monitor
<code>ReentrantLock</code>	explicit/reentrant lock
<code>race</code>	bad interleaving result
<code>deadlock</code>	circular waiting
<code>starvation</code>	resource alamama
<code>livelock</code>	active, no progress
<code>parallel safe</code>	stateless + non-interfering + associative
<code>ordered output</code>	<code>forEachOrdered</code> / <code>findFirst</code>